SCS 3204: AUTOMATA THEORY

Lecture VIII: Kleene's Theorem

1

Kleene's Theorem

- Any language that can be defined by:

  a regular expression

  or finite automaton

  or transition graph

  can be defined by all three methods.

  => all three of these methods of defining languages are equivalent.
- This theorem is the most important and fundamental result in the Theory of Finite Automata

2

Proof of Kleene's Theorem

- We wish to prove that the languages defined by FAs, TGs and REs are equivalent. How?
- Three step proof:

  Part 1: Every language that can be defined by a FA can also be defined by a TG
  Part 2: Every language that can be defined by a TG can also be defined by a RE
  Part 3: Every language that can be defined by a RE (regular expression) can also be defined by a FA

3

Proof of Part 3

Proof by recursive definition and constructive algorithm at the same time:

Remember!! The recursive definition of regular expressions

Rule 1: Every letter of Σ is a regular expression. Λ itself is a regular expression.

Rule 2: If r_1 and r_2 are regular expressions then so are (r_1) r_1r_2 r_1+r_2 r_1^*

Rule 3: Nothing else is a regular expression (exclusion rule)

=> as we are building up a RE, we could at the same time be building up a FA that accepts the same language.

4

Proof by Recursive definition

Rule 1: There is a FA that accepts any particular letter of the alphabet. There is a FA that accepts only the word Λ.

Rule 2: If there is a FA called $\mathbf{FA_1}$ that accepts the language defined by the RE $\mathbf{r_1}$ and there is a FA called $\mathbf{FA_2}$ that accepts the language defined by the RE $\mathbf{r_2}$, then there is a FA called $\mathbf{FA_3}$ that accepts the language defined by the RE $\mathbf{(r_1 + r_2)}$.

Rule 3: If there is a FA called $\mathbf{FA_1}$ that accepts the language defined by the RE $\mathbf{r_1}$ and there is a FA called $\mathbf{FA_2}$ that accepts the language defined by the RE $\mathbf{r_2}$, then there is a FA called $\mathbf{FA_3}$ that accepts the language defined by the concatenation $\mathbf{(r_1r_2)}$.

Rule 4: If $\mathbf{r}$ is a RE and $\mathbf{FA_1}$ is a FA that accepts exactly the language defined by $\mathbf{r}$, then there is a FA called $\mathbf{FA_2}$ that will accept exactly the language defined by $\mathbf{r^*}$.

5

Rule 1

- A FA that accepts Λ?
- A FA that accepts a particular letter of the alphabet ?

6

Rule 2 (by constructive algorithm)

- The language of the new machine will be the union of the two languages.
- Guiding principle: The new machine will simultaneously keep track of where the input would be if it were running on FA_1 and where the input would be if it were running on FA_2.
- General description of Algorithm:
  - starting with two machines FA_1 and FA_2 with states $x_1, x_2, x_3...$ and $y_1, y_2, y_3...$ respectively
  - build a new machine FA_3 with states $z_1, z_2, z_3...$ where each z is of the form "x_i or y_j" for some i or j.
  - If either the x part or the y part is a final state, then the corresponding z state is a final state.
  - To go from one z to another by reading a letter from the input string, we see what happens to the x part and to the y part and go to the new z accordingly.

7

Examples

- Suppose we have the machine FA_1 which accepts the language of words over the alphabet {a,b} that have a double a somewhere in them, and FA_2 which accepts the language EVEN-EVEN over the same alphabet. Build FA_3 which is a union of FA_1 and FA_2, which accepts all words that either have a double a in them or are in EVEN-EVEN.
- Let FA_1 be the machine that accepts all words that end in a, and let FA_2 be the machine that accepts all words ending in b. Build FA_3 which is a union on FA_1 and FA_2.

8

Rule 3 (by constructive algorithm)

- The language of the new machine will be the concatenation of the two languages.
- Guiding principle: The new machine will incorporate a new state with a dual identity: it may mean that we have reached a final state for the first half of the input (a word in FA_1) and we then cross over to FA_2, or else it is merely another state in FA_1 that the string must pass through to get eventually to its final state in FA_1.
- General description of Algorithm:
  - starting with two machines FA_1 and FA_2 with states $x_1, x_2, x_3...$ and $y_1, y_2, y_3....$ respectively. Build a new machine FA_3 with states $z_1, z_2, z_3...$
  - first make a z state for every nonfinal x state in FA_1.
  - For each final state in FA_1 we establish a z state that expresses the options that we are continuing on FA_1 or are beginning on FA_2.
  - From there, establish z states for all situations of the form: we are in x_i continuing on FA_1 or we have just started y_1 about to continue on FA_2 or are in y_j continuing on FA_2.

9

Examples

- Suppose we have the machine FA_1 which accepts the language of words over the alphabet {a,b} with a double a in them, and FA_2 which accepts the language of all words that end in the letter b. Build FA_3 which is a concatenation of FA_1 and FA_2, which accepts exactly those strings that have a front section with a double a followed by a back section that ends in b.
- Let FA_1 be the machine that accepts all words that do not contain the substring aa, and let FA_2 be the machine that accepts all words with an odd number of letters. Build FA_3 which is a concatenation of FA_1 and FA_2.

10

Rule 4 (by constructive algorithm)

Rule 4: If **r** is a RE and **FA_1** is a FA that accepts exactly the language defined by **r**, then there is a FA called **FA_2** that will accept exactly the language defined by **r***.

- The language defined by r* must always contain the null word! => we must therefore indicate that the start state is also a final state. This is quite a change from the language of r where strings returning to the start state may not have been accepted before! Some words in r* may not have been accepted by FA_1.
- General rule: each z state corresponds to some collection of x states. We must remember that each time we reach a final state, it is possible that we have to start over again at x_1.

11

Example

- Given that FA_1 is a machine that accepts the language defined by r where r = aa*bb*, define FA_2 which accepts r*.
- Given that FA_1 is a machine that accepts the language L_1 of all words with an odd number of b's, define FA_2 which accepts L_1*.
- Given that FA_1 is a machine that accepts the language defined by r where r = a* + aa*b, define FA_2 which accepts r*.

12

Summary of converting RE to FA

- Step by step
- Using a simple algorithm
- We have demonstrated that any regular expression can be converted into an equivalent finite automaton
- Note that the finite automaton does not have to be efficient, it just has to be correct

Summary of Kleene's Theorem

- Using definitions and constructive algorithms we have shown that:
- Every FA is also a TG
- Every TG has an equivalent RE
- Every RE has an equivalent FA, and therefore
- FAs, TGs and REs have equivalent language definition power.

Summary of Kleene's Theorem

- Therefore Kleene's theorem is true
- We choose to use RE, TG or FA for convenience but not because one is more powerful for defining languages than another!
- They have the same expressive power, and the same limitations.

Exercises

$\sum = \{a,b\}$

Let R_1 be the regular expression that defines the language accepted by FA_1 which is a machine that accepts all words that end in a, and let R_2 be the regular expression that defines the language accepted by FA_2 which is a machine that accepts all words with an odd number of letters.

1. Build FA_3 which is defined as R_1+R_2.
2. Build FA_3 which is defined as R_1R_2.
3. Build FA2 which is defined as R_1^*
4. Build FA2 which is defined as R_2^*